

Individual Assignment 1 Report

Software Development and DevOps

Alejandro Zapata

1. Introduction

This project is a single-process web app that helps a group of friends agree on dates for a trip and then organize it. Repository: github.com/TheAleZap/devops_assignment_1.

The problem. Group trips usually stall at the first question: when can everyone actually go? Availability is scattered across group chats, decisions never close, and nobody knows who is handling what. Expense-splitting apps already exist, so this project deliberately focuses on the step before money, matching dates.

Stakeholders. The primary user is the trip organizer in a friend group of 4–10 people, who creates a date poll and shares a link. The secondary users are the other friends, who mark their free days and claim tasks without creating an account. In Assignment 2 the course's deployment pipeline becomes a third stakeholder, which is why the app follows the \$7 container contract.

Scope. The app has two independent feature domains, both persisted in SQLite:

Domain	Responsibility	Tables
Scheduling	Date polls, each person's free days, and ranking of the best N-day windows	date_polls, availability
Trips	Trips, members, a task board with claim and complete rules, and confirming dates from a poll	trips, members, tasks

Out of scope by decision: general polls for destination or activities (ADR-5), a day-by-day itinerary, user accounts, payments, and chat or booking integrations.

2. Development process (SDLC)

I used an **incremental model run as a Kanban-style daily flow**: every day ended with a small working increment, built on its own feature branch and merged into `main` through a pull request.

Why this model. It was a one-person project with a fixed one-week deadline, a fixed minimum scope (two domains, SQLite, tests) and uncertain effort per feature. Waterfall was rejected because it pushes testing and integration to the end, which is the riskiest place to discover problems with seven days in total. Scrum was rejected because sprints, roles and ceremonies add overhead for a team of one. Daily increments also matched the assignment's requirement of commits on at least six distinct days.

SMART goals (set on 28 September).

1. Deliver two independently modular feature domains, Scheduling and Trips, both reading and writing through SQLite, by 4 October.
2. Reach at least 70% test coverage on core business logic, measured with pytest-cov, by 4 October.
3. Make at least 12 meaningful commits across at least 6 distinct days, with no day above 40% of the total.
4. Record each architecture decision in `ADR.md` when it is made, across at least 3 distinct commit dates.
5. Pass all ten §7 deployment-contract checks from a fresh clone before submission.

How I followed it in practice.

Day	Planned increment	What actually happened
Mon 28 Sep	Scope, README, framework decision	README and ADR-1 merged; AI log started
Tue 29 Sep	Skeleton: entry point, config, SQLite, routers	Done; an empty <code>config.py</code> was committed and fixed the same day
Wed 30 Sep	ADR-2, schema, ADR-3, availability endpoints	Done; a branch created from the wrong base caused a merge to rescue the schema commits
Thu 1 Oct	Date-window algorithm and tests	Done, plus the calendar frontend pulled forward from Friday
Fri 2 Oct	Trips domain: members, task board, date confirmation	Members and task board done; my laptop died before the date-confirmation code was written
Sat 3 Oct	Seam, tests, ADR-4, ADR-5, README, report	Seam and tests done; most planned documentation slipped
Sun 4 Oct	Final review and small fixes	ADR-4, ADR-5, README, fresh-clone verification and this report

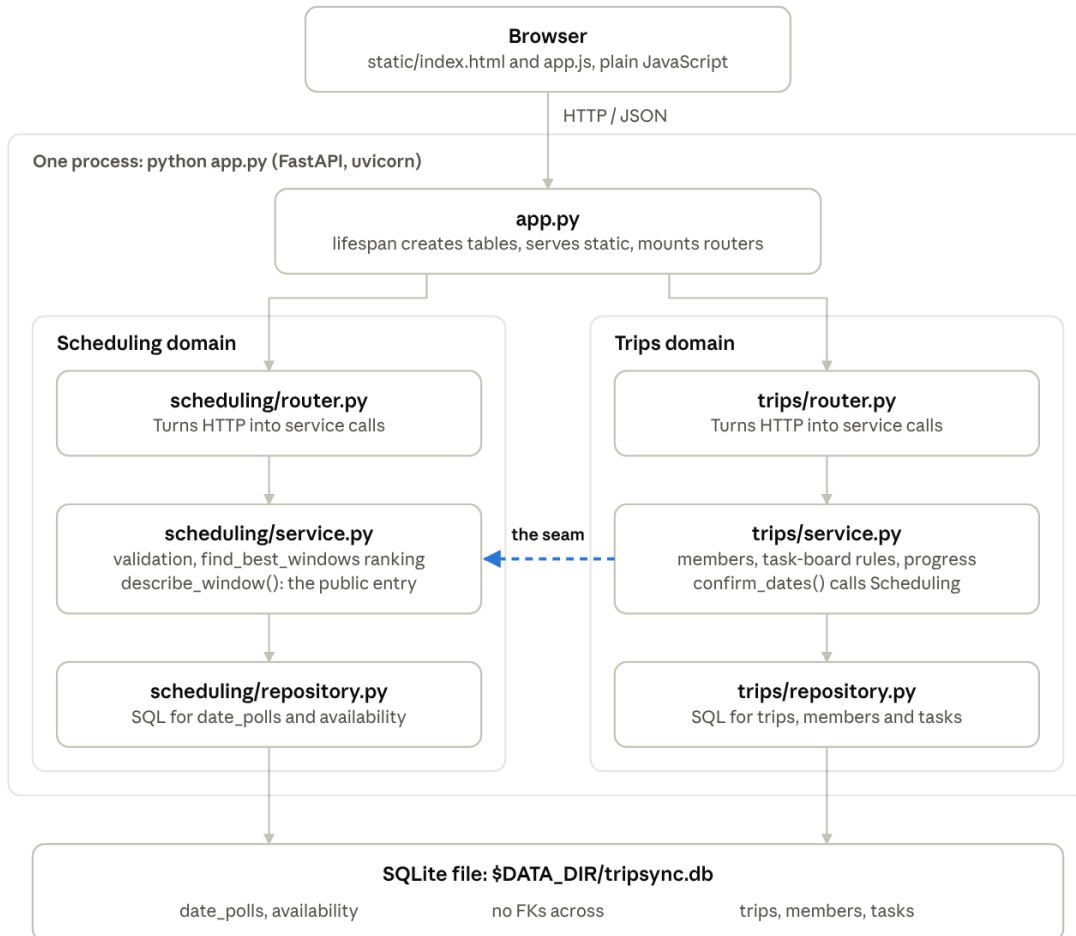
Where reality diverged, and what I learned. Three process slips are visible in the history and I left them there rather than rewriting pushed commits. The task-board code landed in a commit labelled as date confirmation; the real date confirmation followed in its own commit, and the pull-request description explains it. Resolving a merge conflict in `AI_USAGE.md` silently dropped six log rows; I noticed it in the merge diff and restored them from Git history. The general polls I proposed were cut mid-week and documented as ADR-5. The main lesson is that the plan held for the code but not for the

documentation: writing docs alongside each increment, not at the end, would have kept Sunday as a pure buffer.

3. Architecture overview

TripSync runs as one FastAPI process in which each domain has its own router, service and repository, and Trips reaches Scheduling only through `describe_window()` in Scheduling's service layer.

Trips reaches Scheduling only through its service layer



TripSync architecture · one process, two domains, one seam

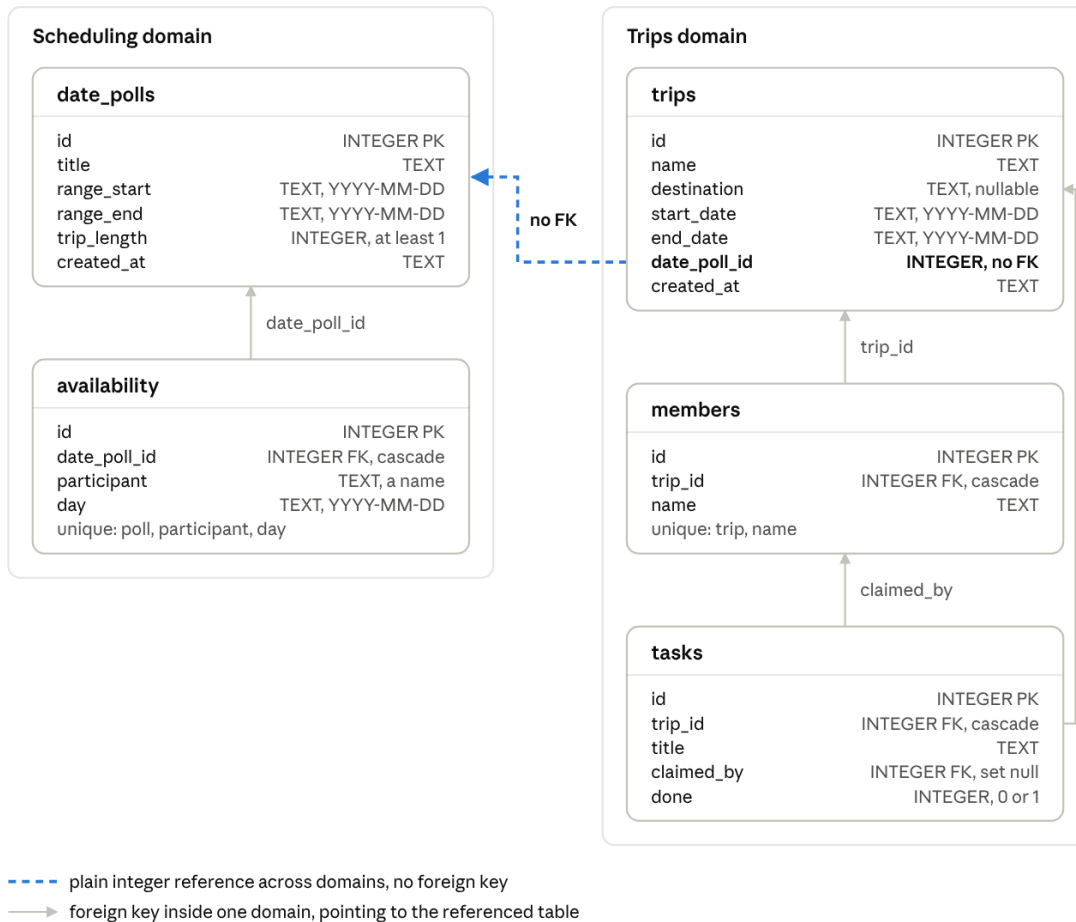
- **Layers.** Routers only translate HTTP into service calls and errors into status codes. Services hold every business rule and are what the tests cover. Repositories hold all SQL, using `?` placeholders and one transaction per write.
- **Startup.** The `lifespan` hook in `app.py` creates `DATA_DIR`, the database file and each domain's tables. Each repository owns its own schema, so `db.py` never imports a domain.
- **The seam.** `confirm_dates()` in Trips calls `scheduling_service.describe_window()` and translates `SchedulingError` into its own `TripsError`. The dependency runs one way, so in Assignment 2 this single call becomes one HTTP request between two services (ADR-2).

- **Frontend.** Plain HTML, CSS and JavaScript served by the same process. `app.js` keeps one state object and re-renders from it, handles dates in UTC, and writes user input with `textContent` to prevent XSS.

4. Database model

The schema has five tables in one SQLite file, two owned by Scheduling and three by Trips, and no foreign key crosses the domain boundary (ADR-3).

Foreign keys stay inside each domain



Database schema · 5 tables in one SQLite file, matching ADR-3

- **No cross-domain foreign keys.** `trips.date_poll_id` is a plain integer and `availability.participant` is a name, not a `members.id`. Each domain can therefore move to its own database in Assignment 2 without breaking constraints. The cost is that the application, not the database, checks those references.
- **One row per free day.** Storing days instead of date ranges turns window ranking into simple counting. At about 10 people and 90 days, a poll stays under 900 rows.

- **Dates as ISO text.** SQLite has no date type, and YYYY-MM-DD strings sort and compare correctly as text.
- **Rules the database enforces.** Unique constraints stop a person marking the same day twice and duplicate members on a trip. Deleting a trip cascades to its members and tasks, and removing a member unclaims their tasks instead of deleting them.
- **Who creates the tables.** Each domain's repository.py owns its CREATE TABLE IF NOT EXISTS statements, run at startup, so startup is safe to repeat.

5. Testing

The suite has **40 passing tests with 76% total coverage**, and every service and repository module is at 100%. The 24% gap is the two routers, which are deliberately untested (ADR-4).

Module	Statements	Missed	Coverage
scheduling/service.py	73	0	100%
scheduling/repository.py	32	0	100%
scheduling/router.py	36	36	0%
trips/service.py	87	0	100%
trips/repository.py	63	0	100%
trips/router.py	46	46	0%
Total	337	82	76%

What I prioritized. The tests target the business rules, because that is where a silent mistake would hurt users:

- **Date-window ranking:** one test per ranking rule (attendance first, fewest missed days second, earliest start last), plus boundaries such as a trip that exactly fills the range and the last window ending on the range end.
- **Validation:** inverted ranges, trips longer than the range, and days outside the poll.
- **Task board rules:** each rule tested from both sides, for example another member cannot take over a claimed task, but the same member can claim it again.
- **The seam:** date confirmation is tested against a real poll created through the Scheduling service, so the two domains are tested together.

How the database is isolated. The temp_db fixture uses pytest's tmp_path and monkeypatch to point config.DB_PATH at a fresh temporary file for every test. Tests never touch data/tripsync.db, and each starts from an empty schema.

What I left thinner, and why. Routers only translate HTTP requests into service calls and errors into status codes, and I checked them manually with curl. Testing them would need FastAPI's TestClient

and one more dependency. The accepted risk is that a wrong status code would not be caught automatically, so router tests are the first thing to add next.

Command: `python -m pytest --cov=scheduling --cov=trips --cov-report=term-missing`

6. Deployment contract verification

All ten §7 rules passed on 4 October, checked on a fresh clone from GitHub into `/tmp`, with a new virtual environment and nothing from my working copy.

Rule	Requirement	How it was verified
7.1	One start command	<code>python app.py</code> started the app from the clean clone
7.2	Bind to <code>0.0.0.0</code>	Startup log: <code>Uvicorn running on http://0.0.0.0:8000</code>
7.3	Port from one env variable	<code>PORT=9001</code> served <code>/health</code> on port 9001
7.4	No interactive setup	No prompts; the lifespan hook creates the folder, database and tables
7.5	One documented SQLite path	<code>data/tripsync.db</code> by default; <code>DATA_DIR=/tmp/tripsync-clone-data</code> moved it there
7.6	No external runtime services	Only Python and four pip packages
7.7	One dependency manifest	Only <code>requirements.txt</code> in the repository root
7.8	No Docker or CI dependency	No Dockerfile, compose file or <code>.github/</code> folder
7.9	Env-only configuration	No <code>.env</code> file exists or is read
7.10	Ready within seconds	“Application startup complete” appeared almost immediately

The same clone also ran the full test suite: 40 passed, 76% coverage.

7. Setup and README

The repository README holds the full setup, configuration, API and test instructions; the short version is below. It requires Python 3.12 or newer and was developed on Python 3.14.

```
git clone https://github.com/TheAleZap/devops_assignment_1.git
cd devops_assignment_1
```

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

The app runs at <http://localhost:8000>, with interactive API docs at </docs>.

Variable	Default	Purpose
PORT	8000	Port the server listens on, always bound to 0.0.0.0
DATA_DIR	data	Folder for the database file tripsync.db

Coverage command: `python -m pytest --cov=scheduling --cov=trips --cov-report=term-missing`. It uses `python -m pytest` so that the pytest inside the virtual environment runs, not a global one without the coverage plugin.

8. AI disclosure statement

I acknowledge the use of Claude (Anthropic) to brainstorm and stress-test the project idea, compare architecture options, generate code for the FastAPI backend, SQLite repositories, date-window algorithm, frontend and unit tests, debug Git, virtual-environment and test failures, and draft the ADR entries, README and this report. I also used ChatGPT to rewrite some of my own text more clearly. The prompts used include “Explain the architecture options I have and I will decide”, “Design how to rank trip windows, then implement the algorithm”, “Write unit tests for scheduling validation and the date-window ranking” and “Debug `NameError: db is not defined in scheduling/repository.py`”. The output of these prompts was used to build and test the application, which I ran, verified and debugged myself, and as drafts for the documentation, which I reviewed and edited; the design decisions, such as the FastAPI and JavaScript architecture, ranking by attendance before convenience, and cutting general polls, were mine. Every meaningful interaction is logged with my own explanation of how the code works in `AI_USAGE.md`.